

CCPROG1 (for S24B) MP Specifications – Part 1 of 3
Taylor Series Expansion of Sine and Cosine Trigonometric Functions
Part 1 is 20% of the MP Grade

I. INTRODUCTION

The 1st part of the machine problem is concerned with the (approximation of the) trigonometric functions **cos x** and **sin x** (cosine and sine) where x is a parameter representing the angle in **radians**.*

The Taylor series expansions of these trigonometric functions are:

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$$
$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots$$

The exclamation mark (!) denotes the factorial symbol. Note that the ellipses indicate that there may be more terms to follow in the series. In the equations shown above, the approximation explicitly shows only 4 terms for each function. In the case of the cosine function, the first term is 1, the second term is $-\frac{x^2}{2!}$, the third term is $\frac{x^4}{4!}$, and the fourth term is $-\frac{x^6}{6!}$. Note that increasing the number of terms improves the accuracy of the approximation.

The Taylor series shown above is quite interesting from the viewpoint of programming. To implement the series, the programmer should apply his/her knowledge of:

- mathematical expressions
- conditional control structures
- counters
- accumulators, and
- loop control structures

The objective, therefore, of this machine problem is for you to demonstrate your knowledge of these concepts.

II. REQUIREMENTS

Your task is to write a C program that computes the Taylor series approximation of the **cos x** and **sin x**. The [angle x is double data type value](#).

For Part 1, you are required to write/implement all codes using `main()` only. DO NOT define any other function aside from `main()`.

* It is assumed that you are familiar with the concepts of these trigonometric functions since they should have been covered as part of your mathematics subjects in your high school or first year college years.

You are allowed to use only standard C99 programming concepts/constructs discussed in class (and in the handouts). Non-standard C constructs (for example, the `clrscr()` or `gotoxy()` functions) are prohibited; usage of such constructs will automatically result into a grade of 0.0 for this particular requirement.

The program DOES NOT require any input from the user. Thus, it is incorrect to call `scanf()` function.

The output of the program is a tabular list of values of the angles in both degrees and radian units, and the corresponding cosine and sine of the angles. Note that your program should convert angles from degrees to radians.

To test the computations for both positive and negative angles, approximate the cosine and sine from -180 degrees to 180 degrees in increments of 5 degrees. The program's output should follow the format specified in the table below.

<your IDNUMBER>			
<one empty line>			
-180.00	<value>	<value>	<value>
-175.00	:	:	:
-170.00	:	:	:
:	:	:	:
:	:	:	:
:	:	:	:
-10.00	:	:	:
-5.00	:	:	:
0.00	<value>	<value>	<value>
5.00	:	:	:
10.00	:	:	:
:	:	:	:
:	:	:	:
:	:	:	:
170.00	:	:	:
175.00	:	:	:
180.00	<value>	<value>	<value>

The first line of output should be your ID number followed by an empty line. Thereafter there should be four columns of tabular output.

The values in the first column show the angle in degrees, the second column is the angle in radians, the third column is for the cosine of the angle and the last column is for the sine of the angle.

Indicate 2 digits after the decimal point for the value of angle x in degrees, and 12 digits after the decimal point for the values of x in radians, and for the corresponding cosine and sine values.

It is imperative that you follow the format because the result of your program will be checked automatically using another program. Point deductions will be applied for not following the format.

Three example results (using different number of terms) will be supplied to you together with this document for your reference.

IMPORTANT NOTE #1:

HONOR CODE and ACADEMIC HONESTY POLICY. You are honest – a person with integrity. You are expected to produce/and submit a project that you DID on your own, and not by other people. Conversely, it means that you DID NOT contribute in any way to the work submitted by another student; this includes discussions about the algorithms/solutions to the problem with another individual (except with your teacher). It also means that you did not search the internet for actual program codes that solve the machine problem.

Violation of the academic honesty policy, i.e., cheating is a major offense and will result in a FINAL GRADE of 0.0.

If you have any question/clarification regarding the problem, please feel free to approach or email me. Do NOT ask any other person.

IMPORTANT NOTE #2:

DOCUMENTATION and CODING STYLE. Document your codes using in-line comments. Use variable names that are suggestive of their usage. Your programs should exhibit readability. Follow as much as possible the Linux Kernel coding style as described in the following link [C Coding Style - GNOME Developer Documentation](#)

IMPORTANT NOTE #3:

Include a **PREAMBLE** in the form of comments in the first few lines of your C source program indicating the following for identification purposes

```
/*  CCPROG1 Machine Problem Part 1                */
/*  <LASTNAME>, <FIRSTNAME>                        */
/*  <SECTION>                                       */
/*  <short description of what this program is all about */
```

IMPORTANT NOTE #4:

Include the following macro definitions in your program:

```
#define PI      3.141592653589
#define NUMBER_OF_TERMS    10
```

The macro **PI** indicates the value of math constant π (PI) which you will need to convert the angles from degrees to radian units. The macro **NUMBER_OF_TERMS** indicates the number of terms to be used in the approximation. You need to use this macro in your program. Later on, you might want to try changing it to other values, for example, 5 or 10 or 15 to see the improvement in the approximation. However, before you submit your source code, make sure that **NUMBER_OF_TERMS** is set to **10** (not some other value).

The only header file that should be included is `stdio.h`. Other header files are NOT allowed.

IMPORTANT NOTE #5:

To promote and learn “good” programming practices

- do NOT use global variables (i.e., variables declared outside any function),
- do NOT use `goto` statements (i.e., to jump from code segments to code segments),
- do NOT use `break` statement to exit a block other than switch blocks,
- do NOT use `return` statement or `exit` statement to **prematurely** terminate a loop or function or program,

Non-compliance will result to corresponding point deductions.

III. DELIVERABLES AND SUBMISSION DEADLINES

You need to submit THREE items, namely:

1. Your C program source code via Canvas - use your last name, followed by an underscore, followed by mp1 as filename. For example, if your last name is SANTOS, your file should be named **SANTOS_mp1.c**.

The softcopy of your source code should be submitted via the Canvas submission page for MP1. Submission via email will NOT be accepted as Canvas is the official repository of all student submissions (not my email inbox).

WARNING: Make sure that you comply with the submission deadline (date and time). Late submission will still be accepted up to 3 days from the submission deadline. Note that an incremental deduction scheme will be applied to late submissions to be fair with students who submitted on time. The deduction scheme is explained in the Canvas submission page. It is described below for your immediate reference:

Late by 1 day or a fraction thereof: 10% deduction

Late by 2 days or a fraction thereof: 10% (for the 1st day) + 20% (for the 2nd day) for a total of 30% deduction.

Late by 3 days or a fraction thereof: 10% (for the 1st day) + 20% (for the 2nd day) + 30% (for the 3rd day) for a total of 60% deduction.

No submission will be allowed 3 days beyond the submission.

2. Accomplished MP submission checklist (print and accomplish the document named “mp1_checklist.pdf”).
3. On short-bond paper, the hardcopy/printout of your: 3 sample output corresponding to **NUMBER_OF_TERMS** equal to 5, 10 and 15 respectively.

For items 2 and 3: **staple** the MP submission checklist on top of sample output hardcopies. Submit these within the **first 10 minutes of our in-person class nearest the date after the MP1 Canvas submission deadline.**

IV. TESTING, CHECKING and GRADING SCHEME

Your source code will be compiled, linked and tested. **BLACK-BOX TESTING will be used to check your submission.**

Note that by default you will receive a perfect grade of 100%. It will remain 100% if there are no programming errors and that you complied properly with all the specifications.

Deductions will be applied based on the following:

- Programming error – syntax error and/or logical error will result into a major deduction; see table below
- Non-compliance with specification – will result in a point minor deduction **for each** infraction; for example: incorrect filename, incorrect printout, etc.
- Bad programming style – will result to minor deductions for source codes that DO NOT follow recommended “good” programming practices thus resulting to difficult-to-read codes due for example to incomprehensible variable name, bad indentation, etc.

A simple grading scheme (no minor deduction applied yet) is indicated in the following table.

Grade	Remarks
100%	Correct results
50%	Half-correct only (for example: cosine is correct but sine is not
10%	Error/s in compilation, linking or execution (logical error) <i>(note that 10% is essentially just a “consolation” point)</i>

As mentioned above, the output of your program will be checked automatically using another program. I will be browsing your program code but will not try to check it in detail for semantics nor point out the error if there is something wrong with your solution. *In case your program turns out NOT to be 100% semantically correct, you still have a lot of chance to fix it in Part 2 of the machine problem.*

V. REFERENCE

Check out the following site for Taylor series expansions of other related trigonometric functions.

http://www.efunda.com/math/taylor_series/trig.cfm

***** End of the Machine Problem Specifications *****

Last Note: Consult me immediately if you have any clarification, question or problem regarding this academic activity. Enjoy! ☺☺☺